

c0a2502080 / ProjExD_Group15

🔍

🔒

+

🕒

🔗

📄

🏠

<> Code

🔗 Pull requests

🔄 Agents

🎬 Actions

📁 Projects

📖 Wiki

🛡️ Security and quality

📈 Insights

⚙️ Settings

ProjExD_Group15 / README.md

...

 c0a2502080 README追記

bc354c4 · 17 minutes ago

🕒

PreviewCodeBlame31 lines (24 loc) · 1.29 KB

🔗🔒Raw📄📥✎⌵☰

シューティングこうかтон

実行環境の必要条件

- python >= 3.10
- pygame >= 2.1
- 必要なものがあれば追記してください（非推奨）

ゲームの概要

- 主人公キャラクター（こうかтон）をマウスで操作し、敵UFOを打つシューティングゲーム
- 参考URL：[サイトタイトル](#)

ゲームの遊び方

- 矢印キーでこうかтонを操作し、スペースキーで射撃する。移動は十字キー。
-

ゲームの実装

共通基本機能

- 第4回講義の演習課題

分担追加機能

- スクロール：敵及び背景を動かす。（上下もあり）：C0A25052
- 回復手段:C0A25020 体力が減ったらマックスになるまで左からランダムに回復アイテムが流れてくる

- チャージビーム（ためてビームをで書くするなど） *加速エリア *ボスの実装（倒したらクリア画面？）：C0C25018 *弾数の実装（初期弾数 5 発、敵を倒すたびに 5 発装填）：C0A25116

メモ

- クラス内の変数は、すべて、「get_変数名」という名前のメソッドを介してアクセスするように設計してある
- すべてのクラスに関係する関数は、クラスの外で定義してある

 c0a2502080 / ProjExD_Group15





 Code  Pull requests  Agents  Actions  Projects  Wiki

 Security and quality  Insights  Settings

ProjExD_Group15 / musou_kokaton.py 

...

 c0a2502080 回復機能実装

f1613b1 · 25 minutes ago



```
1  import math
2  import os
3  import random
4  import sys
5  import time
6  import pygame as pg
7
8  WIDTH = 1100 # ゲームウィンドウの幅
9  HEIGHT = 650 # ゲームウィンドウの高さ
10 os.chdir(os.path.dirname(os.path.abspath(__file__)))
11
12
13  def check_bound(obj_rct: pg.Rect) -> tuple[bool, bool]:
14      """
15      オブジェクトが画面内or画面外を判定し、真値値タプルを返す関数
16      引数：こうかとんや爆弾、ビームなどのRect
17      戻り値：横方向、縦方向のはみ出し判定結果（画面内：True／画面外：False）
18      """
19      yoko, tate = True, True
20      if obj_rct.left < 0 or WIDTH < obj_rct.right:
21          yoko = False
22      if obj_rct.top < 0 or HEIGHT < obj_rct.bottom:
23          tate = False
24      return yoko, tate
25
26
27  def calc_orientation(org: pg.Rect, dst: pg.Rect) -> tuple[float, float]:
28      """
29      orgから見て、dstがどこにあるかを計算し、方向ベクトルをタプルで返す
30      引数1 org：爆弾SurfaceのRect
31      引数2 dst：こうかとんSurfaceのRect
32      戻り値：orgから見たdstの方向ベクトルを表すタプル
33      """
34      x_diff, y_diff = dst.centerx - org.centerx, dst.centery - org.centery
35      norm = math.sqrt(x_diff**2 + y_diff**2)
36      return x_diff / norm, y_diff / norm
37
38
39  class Bird(pg.sprite.Sprite):
40      """
41      ゲームキャラクター（こうかとん）に関するクラス
```

```
42     """
43
44     delta = { # 押下キーと移動量の辞書
45         pg.K_UP: (0, -1),
46         pg.K_DOWN: (0, +1),
47         pg.K_LEFT: (-1, 0),
48         pg.K_RIGHT: (+1, 0),
49     }
50
51     def __init__(self, num: int, xy: tuple[int, int]):
52         """
53         こうかとかん画像Surfaceを生成する
54         引数1 num: こうかとかん画像ファイル名の番号
55         引数2 xy: こうかとかん画像の位置座標タプル
56         """
57         super().__init__()
58         img0 = pg.transform.rotozoom(pg.image.load(f"fig/{num}.png"), 0, 0.9)
59         img = pg.transform.flip(img0, True, False) # デフォルトのこうかとかん
60         self.imgs = {
61             (+1, 0): img, # 右
62             (+1, -1): pg.transform.rotozoom(img, 45, 0.9), # 右上
63             (0, -1): pg.transform.rotozoom(img, 90, 0.9), # 上
64             (-1, -1): pg.transform.rotozoom(img0, -45, 0.9), # 左上
65             (-1, 0): img0, # 左
66             (-1, +1): pg.transform.rotozoom(img0, 45, 0.9), # 左下
67             (0, +1): pg.transform.rotozoom(img, -90, 0.9), # 下
68             (+1, +1): pg.transform.rotozoom(img, -45, 0.9), # 右下
69         }
70         self.dire = (+1, 0)
71         self.image = self.imgs[self.dire]
72         self.rect = self.image.get_rect()
73         self.rect.center = xy
74         self.speed = 10
75         self.state = "normal"
76         self.hyper_life = 0
77
```

ProjExD_Group15 / musou_kokaton.py

↑ Top

Code

Blame

463 lines (393 loc) · 15.7 KB



Raw



```
39     class Bird(pg.sprite.Sprite):
78         def change_img(self, num: int, screen: pg.Surface):
84             self.image = pg.transform.rotozoom(pg.image.load(f"fig/{num}.png"), 0, 0.9)
85             screen.blit(self.image, self.rect)
86
87         def update(self, key_lst: list[bool], screen: pg.Surface, score: "Score"):
88             """
89             押下キーに応じてこうかとかんを移動させる
90             引数1 key_lst: 押下キーの真理値リスト
91             引数2 screen: 画面Surface
92             引数3 score: スコア管理オブジェクト
93             """
94             sum_mv = [0, 0]
95             for k, mv in __class__.delta.items():
96                 if key_lst[k]:
97                     sum_mv[0] += mv[0]
98                     sum_mv[1] += mv[1]
99             self.rect.move_ip(self.speed * sum_mv[0], self.speed * sum_mv[1])
```

```

100         if check_bound(self.rect) != (True, True):
101             self.rect.move_ip(-self.speed * sum_mv[0], -self.speed * sum_mv[1])
102         if not (sum_mv[0] == 0 and sum_mv[1] == 0):
103             self.dire = tuple(sum_mv)
104         if self.state == "hyper":
105             self.hyper_life -= 1
106             self.image = pg.transform.laplacian(self.imgs[self.dire])
107             if self.hyper_life < 0:
108                 self.state = "normal"
109                 self.image = self.imgs[self.dire]
110         else:
111             self.image = self.imgs[self.dire]
112             if key_lst[pg.K_RSHIFT] and score.value > 100:
113                 self.state = "hyper"
114                 self.hyper_life = 500
115                 score.value -= 100
116                 self.image = pg.transform.laplacian(self.imgs[self.dire])
117         screen.blit(self.image, self.rect)
118
119
120     class Bomb(pg.sprite.Sprite):
121         """
122         爆弾に関するクラス
123         """
124
125         colors = [(255, 0, 0), (0, 255, 0), (0, 0, 255), (255, 255, 0), (255, 0, 255), (0, 255, 255)]
126
127     def __init__(self, emy: "Enemy", bird: Bird):
128         """
129         爆弾円Surfaceを生成する
130         引数1 emy: 爆弾を投下する敵機
131         引数2 bird: 攻撃対象のこうかとん
132         """
133         super().__init__()
134         rad = random.randint(10, 50) # 爆弾円の半径: 10以上50以下の乱数
135         self.image = pg.Surface((2 * rad, 2 * rad))
136         color = random.choice(__class__.colors) # 爆弾円の色: クラス変数からランダム選択
137         pg.draw.circle(self.image, color, (rad, rad), rad)
138         self.image.set_colorkey((0, 0, 0))
139         self.rect = self.image.get_rect()
140         # 爆弾を投下するemyから見た攻撃対象のbirdの方向を計算
141         self.vx, self.vy = calc_orientation(emy.rect, bird.rect)
142         self.rect.centerx = emy.rect.centerx
143         self.rect.centery = emy.rect.centery + emy.rect.height // 2
144         self.speed = 6
145         self.state = "active"
146
147     def update(self):
148         """
149         爆弾を速度ベクトルself.vx, self.vyに基づき移動させる
150         引数 screen: 画面Surface
151         """
152         self.rect.move_ip(self.speed * self.vx, self.speed * self.vy)
153         if check_bound(self.rect) != (True, True):
154             self.kill()
155
156
157     class Beam(pg.sprite.Sprite):

```

```
158     """
159     ビームに関するクラス
160     """
161
162     ✓ def __init__(self, bird: Bird):
163         """
164         ビーム画像Surfaceを生成する
165         引数 bird: ビームを放つこうかとん
166         """
167         super().__init__()
168         self.vx, self.vy = bird.dire
169         angle = math.degrees(math.atan2(-self.vy, self.vx))
170         self.image = pg.transform.rotozoom(pg.image.load(f"fig/beam.png"), angle, 1.0)
171         self.vx = math.cos(math.radians(angle))
172         self.vy = -math.sin(math.radians(angle))
173         self.rect = self.image.get_rect()
174         self.rect.centery = bird.rect.centery + bird.rect.height * self.vy
175         self.rect.centerx = bird.rect.centerx + bird.rect.width * self.vx
176         self.speed = 10
177
178     ✓ def update(self):
179         """
180         ビームを速度ベクトルself.vx, self.vyに基づき移動させる
181         引数 screen: 画面Surface
182         """
183         self.rect.move_ip(self.speed * self.vx, self.speed * self.vy)
184         if check_bound(self.rect) != (True, True):
185             self.kill()
186
187
188     ✓ class Explosion(pg.sprite.Sprite):
189         """
190         爆発に関するクラス
191         """
192
193     ✓ def __init__(self, obj: "Bomb|Enemy", life: int):
194         """
195         爆弾が爆発するエフェクトを生成する
196         引数1 obj: 爆発するBombまたは敵機インスタンス
197         引数2 life: 爆発時間
198         """
199         super().__init__()
200         img = pg.image.load(f"fig/explosion.gif")
201         self.imgs = [img, pg.transform.flip(img, 1, 1)]
202         self.image = self.imgs[0]
203         self.rect = self.image.get_rect(center=obj.rect.center)
204         self.life = life
205
206     ✓ def update(self):
207         """
208         爆発時間を1減算した爆発経過時間_lifeに応じて爆発画像を切り替えることで
209         爆発エフェクトを表現する
210         """
211         self.life -= 1
212         self.image = self.imgs[self.life // 10 % 2]
213         if self.life < 0:
214             self.kill()
215
```

```
216
217 ✓ class Enemy(pg.sprite.Sprite):
218     """
219     敵機に関するクラス
220     """
221
222     imgs = [pg.image.load(f"fig/alien{i}.png") for i in range(1, 4)]
223
224 ✓ def __init__(self):
225     super().__init__()
226     self.image = pg.transform.rotozoom(random.choice(__class__.imgs), 0, 0.8)
227     self.rect = self.image.get_rect()
228     self.rect.center = random.randint(0, WIDTH), 0
229     self.vx, self.vy = 0, +6
230     self.bound = random.randint(50, HEIGHT // 2) # 停止位置
231     self.state = "down" # 降下状態or停止状態
232     self.interval = random.randint(50, 300) # 爆弾投下インターバル
233
234 ✓ def update(self):
235     """
236     敵機を速度ベクトルself.vyに基づき移動（降下）させる
237     ランダムに決めた停止位置_boundまで降下したら、_stateを停止状態に変更する
238     引数 screen：画面Surface
239     """
240     if self.rect.centery > self.bound:
241         self.vy = 0
242         self.state = "stop"
243     self.rect.move_ip(self.vx, self.vy)
244
245
246 ✓ class Life:
247 ✓ def __init__(self, num):
248     self.num = num
249     self.image = pg.Surface((40, 40), pg.SRCALPHA)
250
251     pg.draw.circle(self.image, (255, 0, 0), (12, 12), 8)
252     pg.draw.circle(self.image, (255, 0, 0), (28, 12), 8)
253     pg.draw.polygon(self.image, (255, 0, 0), [(4, 16), (20, 36), (36, 16)])
254
255 ✓ def update(self, screen):
256     for i in range(self.num):
257         rect = self.image.get_rect()
258         rect.center = (WIDTH - 50 - i * 45, HEIGHT - 50)
259         screen.blit(self.image, rect)
260
261
262 ✓ class Score:
263     """
264     打ち落とした爆弾、敵機の数スコアとして表示するクラス
265     爆弾：1点
266     敵機：10点
267     """
268
269 ✓ def __init__(self):
270     self.font = pg.font.Font(None, 50)
271     self.color = (0, 0, 255)
272     self.value = 0
273     self.image = self.font.render(f"Score: {self.value}", 0, self.color)
```

```
274         self.rect = self.image.get_rect()
275         self.rect.center = 100, HEIGHT - 50
276
277     def update(self, screen: pg.Surface):
278         self.image = self.font.render(f"Score: {self.value}", 0, self.color)
279         screen.blit(self.image, self.rect)
280
281
282     class EMP:
283         """
284         電磁パルス (EMP) に関するクラス
285         """
286
287     def __init__(self, emys: pg.sprite.Group, bombs: pg.sprite.Group, screen: pg.Surface):
288         emp_surf = pg.Surface((WIDTH, HEIGHT)) # 画面を黄色に
289         emp_surf.set_alpha(128) # 半透明に
290         emp_surf.fill((255, 255, 0))
291         screen.blit(emp_surf, (0, 0))
292         pg.display.update()
293         time.sleep(0.05)
294
295         # 敵機の無効化
296         for emy in emys:
297             emy.interval = float("inf") # 爆弾投下できない
298             try:
299                 emy.image = pg.transform.laplacian(emy.image)
300             except:
301                 pass
302
303         for bomb in bombs: # 爆弾無効か
304             bomb.speed *= 0.5
305             bomb.state = "inactive"
306
307
308     class Gravity(pg.sprite.Sprite):
309         """
310         重力場に関するクラス
311         """
312
313     def __init__(self, life):
314         super().__init__()
315
316         self.image = pg.Surface((WIDTH, HEIGHT))
317         pg.draw.rect(self.image, (0, 0, 0), (0, 0, WIDTH, HEIGHT))
318         self.image.set_alpha(128) # 半透明
319         self.rect = self.image.get_rect()
320
321         self.life = life
322
323     def update(self):
324         self.life -= 1
325         if self.life < 0:
326             self.kill()
327
328     class HealHeart(pg.sprite.Sprite):
329     def __init__(self):
330         super().__init__()
331
```



```
332 # Lifeクラスと同じハート
333 self.image = pg.Surface((40, 40), pg.SRCALPHA)
334
335 pg.draw.circle(self.image, (255, 0, 0), (12, 12), 8)
336 pg.draw.circle(self.image, (255, 0, 0), (28, 12), 8)
337 pg.draw.polygon(self.image, (255, 0, 0), [(4, 16), (20, 36), (36, 16)])
338
339 self.rect = self.image.get_rect()
340 self.rect.x = -40
341 self.rect.y = random.randint(50, HEIGHT - 100)
342
343 self.speed = 5
344
345 def update(self):
346     self.rect.x += self.speed
347     if self.rect.left > WIDTH:
348         self.kill()
349
350 ✓ def main():
351     pg.display.set_caption("真！こうかとん無双")
352     screen = pg.display.set_mode((WIDTH, HEIGHT))
353     bg_img = pg.image.load(f"fig/pg_bg.jpg")
354     score = Score()
355     life = Life(3)
356     bird = Bird(3, (900, 400))
357     bombs = pg.sprite.Group()
358     beams = pg.sprite.Group()
359     exps = pg.sprite.Group()
360     emys = pg.sprite.Group()
361     gravities = pg.sprite.Group()
362     hearts = pg.sprite.Group()
363
364     heal_timer = -1
365     tmr = 0
366     clock = pg.time.Clock()
367     while True:
368         key_lst = pg.key.get_pressed()
369         for event in pg.event.get():
370             if event.type == pg.QUIT:
371                 return 0
372             if event.type == pg.KEYDOWN and event.key == pg.K_SPACE:
373                 beams.add(Beam(bird))
374
375             if event.type == pg.KEYDOWN and event.key == pg.K_e:
376                 if score.value >= 20:
377                     score.value -= 20
378                     EMP(emys, bombs, screen)
379             if event.type == pg.KEYDOWN and event.key == pg.K_RETURN and score.value >= 200:
380                 score.value -= 200
381                 gravities.add(Gravity(400))
382         screen.blit(bg_img, [0, 0])
383
384         if life.num < 3:
385             heal_timer -= 1
386             if heal_timer <= 0:
387                 if len(hearts) == 0: # 画面にハートがないときだけ出現
388                     hearts.add(HealHeart())
389                 heal_timer = 150 # 約3秒ごとに出現
```

```
390
391     if tmr % 200 == 0: # 200フレームに1回, 敵機を出現させる
392         emys.add(Enemy())
393
394     for emy in emys:
395         if emy.state == "stop" and tmr % emy.interval == 0:
396             # 敵機が停止状態に入ったら, intervalに応じて爆弾投下
397             bombs.add(Bomb(emy, bird))
398
399     for emy in pg.sprite.groupcollide(emys, beams, True, True).keys(): # ビームと衝突した敵機リスト
400         exps.add(Explosion(emy, 100)) # 爆発エフェクト
401         score.value += 10 # 10点アップ
402         bird.change_img(6, screen) # こうかとおん喜びエフェクト
403
404     for bomb in pg.sprite.groupcollide(bombs, beams, True, True).keys(): # ビームと衝突した爆弾リスト
405
406         exps.add(Explosion(bomb, 50)) # 爆発エフェクト
407         score.value += 1 # 1点アップ
408
409     # ★追加④ 重力場で敵を破壊
410     for emy in pg.sprite.groupcollide(emys, gravities, True, False).keys():
411         exps.add(Explosion(emy, 100))
412         score.value += 10
413
414     # ★追加⑤ 重力場で爆弾を破壊
415     for bomb in pg.sprite.groupcollide(bombs, gravities, True, False).keys():
416         exps.add(Explosion(bomb, 50))
417         score.value += 1
418
419     for bomb in pg.sprite.spritecollide(bird, bombs, True): # こうかとおん衝突した爆弾リスト
420         if bird.state == "hyper":
421             exps.add(Explosion(bomb, 50))
422             score.value += 1
423         else:
424             life.num -= 1
425             if heal_timer < 0:
426                 heal_timer = 150
427             if life.num <= 0:
428                 bird.change_img(8, screen) # こうかとおん悲しみエフェクト
429                 score.update(screen)
430                 life.update(screen)
431                 pg.display.update()
432                 time.sleep(2)
433                 return
434
435     for heart in pg.sprite.spritecollide(bird, hearts, True):
436         if life.num < 3:
437             life.num += 1
438
439     bird.update(key_lst, screen, score)
440     beams.update()
441     beams.draw(screen)
442     emys.update()
443     emys.draw(screen)
444     bombs.update()
445     bombs.draw(screen)
446     hearts.update()
447     hearts.draw(screen)
```

```
448         gravities.update()
449         gravities.draw(screen)
450         exps.update()
451         exps.draw(screen)
452         score.update(screen)
453         life.update(screen)
454         pg.display.update()
455         tmr += 1
456         clock.tick(50)
457
458
459     if __name__ == "__main__":
460         pg.init()
461         main()
462         pg.quit()
463         sys.exit()
```

c0a2502080 / ProjExD_Group15

🔍

👤

+

🕒

🔗

📄

🏠

<> Code

🔗 Pull requests

🔗 Agents

🔗 Actions

🔗 Projects

🔗 Wiki

🛡️ Security and quality

📈 Insights

⚙️ Settings

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

🔗

base: main

←

compare: C0A25020/回復

✓ **Able to merge.** These branches can be automatically merged.

Discuss and review the changes in this comparison with others. [Learn about pull requests](#)

Create pull request

🔗 2 commits

📄 2 files changed

👤 1 contributor

Commits on Jul 7, 2026

回復機能実装

c0a2502080 committed 25 minutes ago

📄 f1613b1

<>

README追記

c0a2502080 committed 16 minutes ago

📄 bc354c4

<>

Showing 2 changed files with 39 additions and 1 deletion.

Split Unified

2 README.md

19	19	
20	20	### 分担追加機能
21	21	* スクロール：敵及び背景を動かす。（上下もあり）：C0A25052
22		- * 回復手段:C0A25020
	22	+ * 回復手段:C0A25020 体力が減ったらマックスになるまで左からランダムに回復アイテムが流れてくる
23	23	* チャージビーム（ためてビームをで書くするなど）
24	24	*加速エリア
25	25	*ボスの実装（倒したらクリア画面？）：C0C25018

38 musou_kokaton.py

325	325	if self.life < 0:
326	326	self.kill()
327	327	
	328	+ class HealHeart(pg.sprite.Sprite):
	329	+ def __init__(self):
	330	+ super().__init__()
	331	+

	332	+	# Lifeクラスと同じハート
	333	+	self.image = pg.Surface((40, 40), pg.SRCALPHA)
	334	+	
	335	+	pg.draw.circle(self.image, (255, 0, 0), (12, 12), 8)
	336	+	pg.draw.circle(self.image, (255, 0, 0), (28, 12), 8)
	337	+	pg.draw.polygon(self.image, (255, 0, 0), [(4, 16), (20, 36), (36, 16)])
	338	+	
	339	+	self.rect = self.image.get_rect()
	340	+	self.rect.x = -40
	341	+	self.rect.y = random.randint(50, HEIGHT - 100)
	342	+	
	343	+	self.speed = 5
	344	+	
	345	+	def update(self):
	346	+	self.rect.x += self.speed
	347	+	if self.rect.left > WIDTH:
	348	+	self.kill()
328	349		
329	350		def main():
330	351		pg.display.set_caption("真! こうかとん無双")
338	359		exps = pg.sprite.Group()
339	360		emys = pg.sprite.Group()
340	361		gravities = pg.sprite.Group()
	362	+	hearts = pg.sprite.Group()
341	363		
	364	+	heal_timer = -1
342	365		tmr = 0
343	366		clock = pg.time.Clock()
344	367		while True:
358	381		gravities.add(Gravity(400))
359	382		screen.blit(bg_img, [0, 0])
360	383		
	384	+	if life.num < 3:
	385	+	heal_timer -= 1
	386	+	if heal_timer <= 0:
	387	+	if len(hearts) == 0: # 画面にハートがないときだけ出現
	388	+	hearts.add(HealHeart())
	389	+	heal_timer = 150 # 約3秒ごとに出現
	390	+	
361	391		if tmr % 200 == 0: # 200フレームに1回, 敵機を出現させる
362	392		emys.add(Enemy())
363	393		
392	422		score.value += 1
393	423		else:
394	424		life.num -= 1
	425	+	if heal_timer < 0:
	426	+	heal_timer = 150
395	427		if life.num <= 0:
396	428		bird.change_img(8, screen) # こうかとん悲しみエフェクト
397	429		score.update(screen)
398	430		life.update(screen)
399	431		pg.display.update()
400	432		time.sleep(2)
401	433		return
	434	+	
	435	+	for heart in pg.sprite.spritecollide(bird, hearts, True):
	436	+	if life.num < 3:
	437	+	life.num += 1

402	438		
403	439		bird.update(key_lst, screen, score)
404	440		beams.update()
407	443		emys.draw(screen)
408	444		bombs.update()
409	445		bombs.draw(screen)
	446	+	hearts.update()
	447	+	hearts.draw(screen)
410	448		gravities.update()
411	449		gravities.draw(screen)
412	450		exps.update()